



Evaluating the Consistency of Machine Unlearning Verification Under Varying Forgetting Conditions

Shivani Kawade

Prof. Rashmi Chavan², Department of Information Technology, Pillai College of Arts, Commerce and Science,
Panvel, Maharashtra, India

Abstract

Machine unlearning removes a dataset's influence from a trained model, but the evidence used to verify that forgetting occurred doesn't always behave consistently as the forgetting conditions change. This study looks at how consistent standard unlearning verification really is, while keeping the verification setup itself fixed. We ran a controlled CIFAR-10 experiment with a modified ResNet-18, testing GA_FT and SalUn across four conditions that combined forget-set size and forgetting difficulty, with three seeds per setting (24 runs total). Forgetting was assessed using behavioral accuracy, membership inference attack (MIA), Jensen-Shannon (JS) divergence, and centered kernel alignment (CKA), with condition-specific retrained models used as references.

Verification turned out to be strongly condition-dependent. Forget accuracy ranged from 16.60%–92.90% for GA_FT and 58.80%–89.70% for SalUn. Across the 24 runs, evidence signals pointed in mixed directions in 14 runs (58.33%), were tied or neutral in 10 (41.67%), and were never unanimously aligned. These results suggest unlearning verification needs to be read in the context of the forgetting condition, and across multiple complementary signals rather than relying on any single measure. The study offers an empirical baseline for assessing how verification consistency shifts as forget-set size, difficulty, and unlearning method varies.

Keywords: Machine Unlearning, Unlearning Verification, Verification Consistency, Forgetting Conditions, Membership Inference Attack, Jensen-Shannon Divergence, Centered Kernel Alignment, Model Evaluation

I. Introduction

A. Background of Machine Unlearning

Large-scale models are commonly trained on data gathered from individuals, organizations, or a mix of outside sources. Once training wraps up, it's almost inevitable that a need arises to strip out the influence of some particular slice of that data — a person withdraws consent, a new privacy regulation applies, ownership of the data changes

hands, or someone simply requests deletion. The most direct fix is retraining the model from the ground up without the data in question, but once models and datasets reach a certain scale, that route becomes far too costly to be practical. Machine unlearning exists to sidestep that expense: rather than a complete retrain, it tries to remove the footprint of a targeted slice of training data from a model that has already finished training. The data marked for removal is called the “forget set,” and everything else forms the “retain set.” The unlearning procedure then pushes the model’s behavior toward what a model that had never encountered the forget set would look like. Underneath the methods that actually adjust parameters sits a thornier issue, though: how do we confirm the forgetting genuinely worked? A model’s accuracy on the forgotten data might drop while it keeps leaking traces through its outputs, its internal representations, or other behaviors an accuracy check simply wouldn’t catch. Properly assessing unlearning means pulling in evidence from several angles at once instead of trusting a lone number to tell the full story. This study puts that principle to work using CIFAR-10 and a custom ResNet-18. Two unlearning strategies — GA_FT and SalUn — are evaluated under deliberately controlled changes to the forget set’s makeup, to see how well the field’s go-to verification evidence holds up as the forgetting scenario is varied.

B. The Verification Problem

There isn’t a single number that can fully settle whether a model has genuinely forgotten a piece of training data — different verification strategies each end up probing a different facet of the resulting model. Behavioral verification, for one, examines how the model performs on the forget and retain sets: a decline in forget-set accuracy points to a behavioral shift, while retain-set performance reveals what unlearning did to everything the model wasn’t meant to touch.

Privacy-oriented verification comes at the problem from another angle, asking whether forgotten data can still be told apart from data the model never encountered in the first place; here, that’s done through a loss-based membership inference attack (MIA), quantified with ROC-AUC. Output-level verification, in turn, sets the unlearned model’s predictions against those of a reference model, applying Jensen-Shannon (JS) divergence to gauge how far apart the two models’ output distributions land on both forgotten and retained test samples. Representation-level verification pushes further still, comparing internal representations directly: centered kernel alignment (CKA) gets applied to the penultimate-layer representations of the unlearned model against a matching retrained reference model. None of these measures is guaranteed to be capturing the same underlying phenomenon, which raises a methodological question worth confronting head-on: do these established forms of evidence actually line up with each other once the forgetting conditions beneath them begin to shift?

C. Motivation

Most existing work reports verification numbers for a single dataset, model, forgetting setup, and unlearning method at a time. Yet a result drawn from just that one scenario doesn’t necessarily say much about how the same evidence would play out elsewhere. The forget set could hold one class or several, and it could have been easy or hard for the original model to distinguish in the first place. Either factor has the potential to reshape how the unlearned model comes across — behaviorally, from a privacy standpoint, in its outputs, or in its internal representations.

This is the controlled question at the heart of the study:

If the verification evidence is kept fixed, how does what it reports change once the forgetting conditions themselves change?

Rather than tacking on yet another metric, this study instead holds the verification framework steady and directly varies two aspects of the forgetting scenario:

- **Forget-set size** — 10% versus 20% of the training data.
- **Forgetting difficulty** — predefined easy and hard conditions, grounded in how much the original model tends to confuse the relevant classes.

Two unlearning methods, GA_FT and SalUn, are each run across three random seeds, for 24 runs in total — enough to observe how established evidence behaves across a genuine grid of conditions and methods rather than relying on a single snapshot.

D. Research Gap

The literature has already made clear that unlearning verification can pull from several categories of evidence — behavioral performance, membership inference, output-level similarity, and representation-level analysis — each carrying its own well-documented weaknesses. Earlier work has also shown that forgetting difficulty and the makeup of the data shape outcomes, and that different evaluation procedures can read the exact same model quite differently. What's still lacking is a concrete answer to how established verification evidence responds once the forgetting conditions are deliberately changed. Most studies examine one method, one benchmark, one metric, or one procedure under a fixed setup — useful, but it doesn't reveal how a whole set of verification signals behaves as conditions are systematically varied within a single experimental frame. What's needed is a controlled, empirical account of how established machine-unlearning verification evidence responds — and whether it holds steady or turns inconsistent — when forget-set size and forgetting difficulty are systematically varied inside one shared experimental setup.

E. Research Question

This study is organized around a single central question:

RQ1: With the verification evidence held constant, how does its observed response shift as the forgetting conditions change — specifically forget-set size and forgetting difficulty? This gets tested across two unlearning methods, GA_FT and SalUn, using four established evidence types: behavioral accuracy, MIA, JS-based output similarity, and CKA-based representation similarity. The study also asks whether these evidence types point in a shared direction across conditions, or pull apart.

F. Research Objectives

To work through the research question, the study is built around six objectives:

O1. Build controlled forgetting conditions

Construct predefined forgetting scenarios by varying:

- **forget-set size** — 10% and 20% of the training data, and
- **forgetting difficulty** — easy versus hard, based on how much the original model tends to confuse the relevant classes.

That produces four frozen experimental conditions: small-easy, small-hard, large-easy, and large-hard.

O2. Evaluate established verification evidence

Assess the unlearned models using four established evidence types:

- **Behavioral verification** — forget, retain, and overall accuracy.
- **Privacy verification** — MIA ROC-AUC.
- **Output similarity** — JS divergence between unlearned and retrained-reference outputs.
- **Representation similarity** — CKA between unlearned and retrained-reference representations.

O3. Examine how verification responds across conditions

Characterize how the measured signals move as forget-set size and forgetting difficulty vary, while the verification setup itself is held fixed.

O4. Examine cross-signal consistency

Check whether the four forgetting-oriented signals point the same way within a given unlearning run, or come out mixed or neutral instead.

O5. Compare unlearned models against retrained references

Examine how unlearned models diverge from their retrained counterparts, especially on retain-set and overall performance.

O6. Characterize method-dependent behavior

See whether the verification responses differ between GA_FT and SalUn even when the forgetting conditions are held identical.

G. Contributions

This study's contributions come down to: (1) a controlled look at how verification responds to shifting forgetting conditions; (2) a joint examination of behavioral, privacy, output, and representation evidence; (3) deliberate variation of forget-set size and difficulty; (4) an analysis of cross-signal consistency; (5) condition-specific retrained references; and (6) a reading of verification evidence that takes into account the forgetting condition that generated it. In short, the study delivers a controlled, empirical account of how established verification evidence behaves once forget-set size and forgetting difficulty are deliberately varied, while the verification framework itself remains fixed throughout. Rather than examining a single evidence type at a time, the study brings four together: behavioral performance, MIA, JS divergence, and CKA — together painting a fuller picture than any one signal could manage alone. The design splits the forgetting conditions across two factors, forget-set size and forgetting difficulty, yielding four predefined conditions and letting us track how verification evidence responds across a range of scenarios rather than a single one. The study also checks whether the signals move together, come out mixed, or land neutral within the same runs — an empirical read on just how much these established evidence types genuinely agree, or don't. For every condition, a retrained model acts as the comparison baseline. These aren't treated as universal ground truth; they're simply condition-specific reference points that make the behavioral and similarity measurements easier to interpret.

II. Related Work

A. Machine Unlearning

Machine unlearning is fundamentally about reducing, or ideally eliminating outright, the influence that particular training data has on a model that has already finished training. The need for it crops up constantly in ordinary circumstances: a privacy rule takes effect, a data-management policy shifts, a user asks for their data to be removed, or the model's intended purpose changes in a way that makes some of its training data unwelcome. Rather than retraining from zero each time this happens, unlearning methods instead try to adjust the existing model so the

influence of a **forget set** fades while whatever’s useful in the **retain set** stays intact. Researchers have approached this from many different angles — parameter-update tricks, gradient-based methods, data-specific optimization, approximate retraining, and approaches built around particular architectures or learning setups. Benchmarks such as TOFU, MUSE, WMDP, RWKU, UnlearnCanvas, OpenUnlearning, LUME[1]-[7], and others have pushed this empirical work further across different datasets, tasks, and model types. Taken together, they make one point unmistakable: unlearning isn’t something that can be pinned down through a single observable behavior. What counts as evidence for it depends heavily on the kind of information being removed and the shape of the underlying learning problem.

One reference point that shows up repeatedly across this literature is **retraining from scratch without the forget data**. A model built this way gives something concrete to measure against: a rough approximation of what the model would look like had it never encountered the forgotten data at all. It’s worth staying cautious here, though — retraining-based comparison is a reference procedure, not some universal ground truth that resolves every verification question on its own. Differences among the original, unlearned, and retrained models can just as easily stem from optimization randomness, initialization, or ordinary training dynamics as from anything meaningful tied to forgetting, and a great deal depends on exactly which evidence is used to compare them.

B. Evaluation of Machine Unlearning

Simply altering a model’s parameters doesn’t by itself prove that targeted information has actually vanished, which is why the field draws on several distinct kinds of evidence to check whether a model still behaves as though it remembers what it was supposed to forget. **Behavioral evaluation** is the most familiar of these — it looks at how an unlearned model performs on forget and retain data. A drop in forget-set performance is a reasonable indicator that something changed in how the model treats that data, while retain-set performance shows how much useful behavior survived the process. But behavioral shifts alone don’t guarantee the information is fully gone; a model can change what it predicts without genuinely clearing out the internal representations or other traces tied to the forgotten data. Privacy-oriented or inference-based evaluation offers a second angle, built primarily around membership inference attacks (MIAs), which test whether an adversary can distinguish training samples from samples the model never saw at all. In unlearning research specifically, forgotten training samples get compared against matching non-member samples to check whether any trace of “this was in training” still lingers. A number of studies have flagged genuine limitations and sensitivities in MIAs as evaluation tools, which is why an MIA result is best treated as one piece of evidence rather than proof of complete forgetting by itself. A third angle centers on **output-level similarity**. Rather than stopping at classification accuracy, this compares the full predictive distributions produced by two models, including an unlearned model against one retrained without the forget data. Divergence measures such as Jensen-Shannon divergence pick up information plain accuracy tends to miss, making them a useful complement rather than an outright replacement.

C. Verification and Auditing

It’s worth drawing a firm line between **evaluation** and **verification** in this space, even though the two terms often get used almost interchangeably. Evaluation is about measuring how an unlearned model behaves; verification is about deciding whether the gathered evidence actually supports the claim that the requested forgetting took place. Work on unlearning verification and auditing has weighed forgetting claims against behavioral, parameter-level, statistical, and privacy-oriented tests, alongside formal cryptographic mechanisms, making clear that this field extends well beyond

simply checking post-unlearning accuracy. One point the verification literature keeps surfacing is that **different verification criteria can pick up on different facets of forgetting**. A model might display changed predictions while still retaining internal information, or it might pass one evaluation procedure while failing to yield consistent evidence under another. The 2026 survey Towards Reliable Forgetting: A Survey on Machine Unlearning Verification tackles this head-on, drawing distinctions between different forms of verification evidence and arguing that verification shouldn't be flattened into a single number. [25]

D. Representation-Level Verification

A further question worth asking: does output-level behavior really reveal enough about what's happening inside the model? A model can generate predictions that look like successful forgetting while its intermediate representations still hold onto information tied to the forget set. That very possibility is what drove recent representation-focused work to dig into internal model states as an additional source of evidence. An Illusion of Unlearning? Assessing Machine Unlearning Through Internal Representations, in particular, demonstrates that output-level signs of unlearning can sit right alongside discriminative information still living in the hidden representations. Findings like this build a strong case for checking internal representations alongside behavioral and output-level evidence, rather than relying on outputs alone [19]. Representation similarity measures are built to address precisely this gap. CKA quantifies how similar two models' representations actually are; applied to unlearning, comparing an unlearned model's representations against a reference's can surface information that accuracy or membership inference alone doesn't reach.

E. Research Positioning

Pulling the literature together, then: unlearning already gets evaluated through multiple lenses — behavioral, membership inference, output-level, and representation-level. Conclusions about forgetting can shift depending on the evidence, benchmark, data, and setting involved. There's existing research on what makes unlearning difficult and how different methods behave under different scenarios, and recent verification work keeps reinforcing the same message — reliable, complementary evidence matters. What sets this study apart is how it **organizes** those established components experimentally. It holds the verification evidence fixed and systematically varies the conditions under which forgetting happens — specifically, forget-set size and forgetting difficulty across four predefined conditions — while evaluating two unlearning methods with the same verification protocol and multiple random seeds. The resulting behavioral, MIA, output-similarity, and representation-similarity measurements are then read together, to see how they respond as the forgetting conditions shift.

III. Methodology

A. Research Design

This study runs a controlled empirical setup to test how consistent established verification evidence remains as forgetting conditions shift. It isn't proposing a new unlearning algorithm, metric, or formal framework; instead, it puts field-trusted methods and measures through a shared protocol while deliberately varying key characteristics of the forgetting scenario. Two factors drive the design: forget-set size and forgetting difficulty. Crossing these produces four conditions — small-easy, small-hard, large-easy, large-hard. “Small” means one class forgotten; “large” means two. Difficulty itself is derived from how much the original model tends to confuse the relevant classes, fixed before

any unlearning takes place. Two established methods, GA_FT and SalUn, are tested across all four conditions. Each pairing runs three times, with seeds 42, 123, and 2026, for 24 runs in total, letting verification responses be checked against both method choice and seed variation rather than relying on a single run per setup. Every run is checked against the same four evidence types: behavioral performance, MIA, JS divergence, and CKA. The verification evidence itself stays fixed; it's the forgetting conditions that move.

A retrained reference model is built for each of the four conditions, giving something concrete to compare unlearned models against under matching conditions; these are reference points, not proof that unlearning succeeded. Measurements are examined two ways: individually, to see how each signal responds as conditions change, and collectively, to check whether the evidence types agree, mix, or land neutral within the same runs. Descriptive comparisons round this out by isolating what forget-set size and difficulty each contribute on their own.

Put simply, the overall structure runs like this:

Fixed verification evidence → controlled variation of forgetting conditions → established unlearning methods → multi-dimensional verification → cross-signal consistency analysis. That structure is what lets the study actually answer its central question: when the verification evidence is held fixed, how does what it reports change as forget-set size and forgetting difficulty vary?

B. Dataset

The experiments run on **CIFAR-10**, the well-known image classification dataset containing 60,000 color images spread across 10 object classes. It splits into **50,000 training images** and **10,000 test images**, with a clean 6,000 per class. Each image measures **32 × 32 pixels**, across three color channels (RGB). The ten classes are **airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck**. This class-based structure suits what this study needs well — since classes are discrete, whole classes can be pulled out to define the forgetting conditions, keeping a clean boundary between what's being forgotten and what's being retained.

The training set is split **45,000/5,000** into training and validation subsets. The validation portion tracks performance while training the original model and helps select the best checkpoint. Once training finishes, the original model is evaluated on the full CIFAR-10 test set. Every image gets converted into a tensor and normalized using the standard CIFAR-10 per-channel mean and standard deviation:

$$\mu = (0.4914, 0.4822, 0.4465)$$

$$\sigma = (0.2470, 0.2435, 0.2616)$$

This preprocessing carries through every stage — original-model training, unlearning, retraining, and verification — so inputs never change across conditions. For each unlearning run, the training data is split into two disjoint pieces: the **forget set** (F) — the samples whose influence the unlearning procedure is meant to erase — and the **retain set** (R), everything else:

$$D_{\text{train}} = F \cup R, \quad F \cap R = \emptyset$$

The forgetting conditions are built at the class level. **Small** conditions forget just one class — roughly 10% of the training data — while **large** conditions forget two classes, roughly 20%. The test data is split the same way, into forgotten-class and retained-class subsets, so each condition can be evaluated on its own terms.

Here are the four conditions:

Condition	Forget Classes	Approx. Forget Size
Small-Easy	Automobile	10%
Small-Hard	Cat	10%
Large-Easy	Automobile + Truck	20%
Large-Hard	Bird + Cat	20%

Which classes belong to which condition was decided before unlearning began and remained identical across every method and seed, so any differences in verification measurements trace back to the forgetting conditions rather than incidental setup changes. These forget/retain splits carry through everything downstream: unlearning, condition-specific evaluation, membership inference, output- and representation-similarity analysis, and comparisons against the matching retrained reference models.

C. Model Architecture

All experiments run on a customized **ResNet-18**. It's a fitting choice here — residual connections give it ample capacity for CIFAR-10 classification, yet it stays light enough to run repeatedly through unlearning and retraining without turning into a computational bottleneck. The architecture is adapted to CIFAR-10's 32×32 resolution, taking three-channel RGB images and producing predictions across the 10 classes. Residual blocks build up richer visual representations while identity skip connections keep information flowing cleanly through the layers. At the end, the learned representation is mapped down to the ten output classes. In total, the model carries **11,173,962 trainable parameters**. A feature-extraction path supports representation-level verification: beyond class predictions, the network returns a 512-dimensional penultimate feature vector, which feeds the CKA-based representation-similarity analysis. This architecture stays identical across the original model, every unlearned model, and every retrained reference model, so any differences turning up during verification can't be chalked up to structural changes in the model itself.

In effect, the model supplies the study with two outputs it needs:

1. **Class predictions** — feeding behavioral evaluation and output-based verification.
2. **512-dimensional penultimate representations** — feeding the CKA-based representation comparison.

Running every model through the same architecture lets the study watch verification evidence shift while the underlying model structure stays fixed.

D. Forgetting Conditions

To see how verification evidence reacts to different forgetting scenarios, four conditions were constructed by crossing two factors: **forget-set size** and **forgetting difficulty**. These were locked in before any unlearning ran, and applied identically to both GA_FT and SalUn across all three seeds. The first factor — **forget-set size** — separates small from large forgetting scenarios. A small forget set means one CIFAR-10 class, roughly 10% of the training data. A large forget set means two classes, roughly 20%. The second factor — **forgetting difficulty** — separates easy from hard scenarios. Difficulty here isn't arbitrary; it's derived from how the original trained model actually behaves toward the chosen forget classes. How that measure is built is covered separately in Section 4.5.

Crossing these two factors produces a clean 2×2 design:

Condition	Forget-Set Size	Forget Classes	Difficulty
Small-Easy	10%	Automobile	Easy
Small-Hard	10%	Cat	Hard
Large-Easy	20%	Automobile + Truck	Easy
Large-Hard	20%	Bird + Cat	Hard

For small conditions, the forget set consists of every training sample from the one chosen class; for large conditions, every sample from both chosen classes. Whatever remains forms the matching retain set. The breakdown in actual sample counts looks like this:

Condition	Forget Train Samples	Retain Train Samples	Forget Test Samples	Retain Test Samples
Small-Easy	4,488	40,512	1,000	9,000
Small-Hard	4,529	40,471	1,000	9,000
Large-Easy	8,987	36,013	2,000	8,000
Large-Hard	8,997	36,003	2,000	8,000

The numbers land slightly off a clean 10%/20% split because CIFAR-10's training set isn't perfectly class-balanced once validation is removed. These exact partitions remained fixed through every unlearning and reference-model run tied to that condition. These four conditions provide a controlled way to check whether shifts in forget-set size and difficulty line up with shifts in behavioral, membership-inference, output-similarity, or representation-similarity results — and because the protocol never changes, any differences can be read as a response to the forgetting scenario rather than verification noise.

E. Difficulty Construction

Forgetting difficulty in this study is defined purely from how the **original model** — before any unlearning touched it — classifies things. The idea is to lock in difficulty conditions based on the model's own pre-existing confusion patterns, not from anything that happens later during the unlearning experiments themselves.

For a given forget set (F), difficulty is simply the share of forget-set samples the original model misclassifies as **retained classes**:

$$D(F) = \frac{\text{Number of forget-set samples misclassified as retained classes}}{\text{Total number of samples belonging to } F}$$

The higher $D(F)$ climbs, the more forget-set samples get tangled up with classes meant to stay in the model — treated as the harder condition, since the original model already shows strong confusion between what it's about to forget and what it needs to keep. For a single-class forget set, every misclassification counts toward difficulty, since anything outside the forgotten class belongs to the retain set by default. But for a two-class forget set, things call for more care: if the model confuses one forgotten class for the other forgotten class, that doesn't count as a difficulty error — both classes are headed out anyway. Only mix-ups that spill over into a retained class actually add to the numerator. Running the original model's test-set predictions through this yields the following difficulty values for the four conditions:

Condition	Forget Classes	Forget-Set Size	Difficulty
Small-Easy	Automobile	10%	4.80%
Small-Hard	Cat	10%	26.00%
Large-Easy	Automobile + Truck	20%	3.15%

Condition	Forget Classes	Forget-Set Size	Difficulty
Large-Hard	Bird + Cat	20%	18.25%

The class combinations were chosen specifically to produce an easy and a hard version at each forget-set size. And these numbers were **frozen before unlearning started** — they weren't adjusted afterward based on how GA_FT or SalUn happened to perform. Automobile carried a low confusion rate with retained classes, while cat's confusion rate ran noticeably higher; the same pattern holds for automobile+truck versus bird+cat. That contrast defines easy versus hard at both forget-set sizes.

F. Unlearning Methods

Two established approaches are tested: GA_FT and SalUn, chosen because they represent distinct unlearning procedures within the same setup. The aim isn't to introduce or compare new algorithms, but to check whether verification evidence responds consistently across conditions and methods.

1) GA-based Fine-Tuning (GA_FT)

GA_FT runs in two stages: a gradient-ascent pass over the forget set, followed by fine-tuning on the retain set. The forgetting stage updates the model to weaken its learned dependence on forget samples; retain-stage fine-tuning recovers useful predictive behavior that got knocked off in the process.

Conceptually, that breaks into two sequential steps:

1. **Forget-stage optimization** — the model updates on the designated forget set using a gradient-ascent objective.
2. **Retain-stage fine-tuning** — the resulting model gets fine-tuned on the retain set to preserve or recover performance on information that's supposed to stick around.

This GA_FT procedure runs independently across every condition and seed: the forget/retain partitions shift by condition, but the procedure itself never changes.

2) SalUn

SalUn relies on a saliency-based mechanism to identify parameters tied to the forgetting objective, then assigns randomized labels to forget samples before fine-tuning on the retain set.

In practice, this also splits into two stages:

1. **Forget-stage optimization** — forget samples get randomized labels and are run through the forgetting procedure, with parameter saliency guiding the updates.
2. **Retain-stage fine-tuning** — the model is then optimized on the retain set to restore retained-task behavior.

The saliency mechanism stays fixed across every condition. Like GA_FT, SalUn runs independently across all four conditions and three seeds.

3) Common Experimental Treatment

Both methods start from the same original checkpoint. The forgetting condition determines the forget/retain partitions; the method and its configuration remain fixed.

The full pipeline for both methods looks like this:

Original Model → Condition-Specific Forget/Retain Partition → GA_FT or SalUn → Unlearned Model → Verification Evidence

Both methods get run across the same four conditions and three seeds, adding up to 24 unlearning runs total:

2 methods × 4 conditions × 3 seeds = 24 runs The specific optimization settings for GA_FT and SalUn are laid out in Section 5.

G. Retrained Reference Models

To give the unlearned models something concrete to be measured against, **four retrained reference models** were trained — one per predefined forgetting condition. These were trained completely independently of the original model and of the unlearning procedures themselves. For each condition, the relevant forget classes were excluded from the training data from the outset, and a fresh model of the same architecture was trained on what remained — showing what the network looks like having never seen those classes, rather than having had them stripped out afterward. The four reference models line up with the same four conditions used everywhere else:

Condition	Excluded Classes	Reference Training Data
Small-Easy	Automobile	Retained classes
Small-Hard	Cat	Retained classes
Large-Easy	Automobile + Truck	Retained classes
Large-Hard	Bird + Cat	Retained classes

Each one acts as a **condition-specific comparison point**. So an unlearned model built under the Small-Easy condition is compared against the retrained model built under that same Small-Easy condition — never against a reference model from a different forgetting scenario. Condition-matching matters, since what a model naturally looks like shifts with how much retained data it saw and what that data contains. Comparing each unlearned model to a reference trained under the same condition keeps the comparison fair. These retrained models are reference points, not universal ground truth. A retrained model shows one legitimate outcome of training without the forget data, but doesn't on its own prove an unlearned model has fully forgotten anything — hence they're used alongside behavioral, MIA, and similarity evidence, not as a substitute for it. The reference models use the exact same architecture and preprocessing as the original and unlearned models, and all four feed condition-specific comparisons throughout the behavioral and similarity analyses. The role these retrained models play comes down to this: Condition-specific retraining supplies a reference state to compare the matching unlearned model against, without treating that retraining as the final word on whether unlearning succeeded. The full training configuration for these reference models is covered in Section 5.

H. Verification Evidence

Every unlearned model gets checked against four established forms of evidence: behavioral performance, membership inference attack (MIA), output similarity via Jensen-Shannon (JS) divergence, and representation similarity via centered kernel alignment (CKA). These four were chosen because they view the model from different angles: behavioral evaluation checks observable performance; MIA checks whether forgotten samples can be told apart from non-members; JS divergence picks up output-distribution shifts; CKA measures internal feature similarity. The same protocol runs across every unlearned model — both methods, all conditions, all seeds — since the study's whole point is watching how the evidence changes as forgetting conditions move.

The next part takes a closer look at each of the four.

1) Behavioral Verification

Behavioral verification looks at how well the unlearned model performs on the forgotten and retained slices of the test set.

For each condition, the test data splits into:

- **Forget-test set** — samples from the classes marked for forgetting.
- **Retain-test set** — samples from everything else.

Accuracy is calculated separately for each of these, plus once more for the full test set. Forget-set accuracy shows how the model handles classes it should have forgotten; retain-set accuracy shows whether behavior elsewhere held up; overall accuracy rounds out the full test set. These three numbers are kept separate rather than combined into one score, so changes in forgetting behavior and retained-task performance can be tracked independently. Behavioral verification, then, is one piece of the puzzle here, not a standalone verdict on whether unlearning worked.

2) Membership Inference Attack

The MIA analysis checks whether samples from the forget classes can still be picked out based on membership-related model behavior. For each condition, the **members** are forgotten-class training samples, and the **non-members** are forgotten-class test samples. The model's per-sample cross-entropy loss serves as the membership score — flipped to negative loss as the attack score, so bigger numbers mean stronger membership evidence.

The attack is scored using **ROC-AUC**:

$$\text{AUC}_{\text{ROC}} = \int_0^1 \text{TPR}(f) d\text{FPR}(f)$$

An AUC near 0.5 means the attack can barely distinguish members from non-members; higher values mean the groups are more separable. Mean member and non-member loss values are also recorded to aid interpretation. MIA is treated as a privacy-oriented behavioral signal — read alongside the rest of the evidence, not as standalone proof of forgetting.

3) Output Similarity Using Jensen-Shannon Divergence

Output-level similarity is measured with **Jensen-Shannon (JS) divergence**, comparing the predictive probability distributions from the unlearned model against the matching condition-specific retrained reference model. For input x , let $P(x)$ be the unlearned model's distribution and $Q(x)$ the retrained reference's. JS divergence is built off the midpoint distribution:

$$JS(P \parallel Q) = \frac{1}{2} D_{\text{KL}}\left(P \parallel \frac{1}{2}(P+Q)\right) + \frac{1}{2} D_{\text{KL}}\left(Q \parallel \frac{1}{2}(P+Q)\right)$$

where (D_{KL}) is the Kullback-Leibler divergence. Lower JS divergence means the unlearned and reference models are predicting more similarly; higher means they've drifted apart. JS divergence is calculated separately for **forget-test** and **retain-test** samples, so it's possible to see whether output behavior shifts differently on forgotten versus retained data, relative to the matching retrained reference. A small epsilon of (10^{-12}) is used during the probability calculations, purely to sidestep numerical issues from zero probabilities. These JS values are read as output-level similarity evidence — again, not treated as a standalone verdict on whether unlearning succeeded.

4) Representation Similarity Using Centered Kernel Alignment

Representation-level similarity comes from **Centered Kernel Alignment (CKA)**, comparing internal feature representations between the unlearned model and its matching retrained reference. For every sample evaluated, the

512-dimensional penultimate-layer representation described back in Section 4.3 is pulled from both models. Those feature matrices are then compared using linear CKA. CKA measures how similar two representation spaces are once centered. In the linear case, this reduces to the normalized Hilbert-Schmidt independence criterion:

$$\text{CKA}(X, Y) = \frac{\|X^T Y\|_F^2}{\|X^T X\|_F \|Y^T Y\|_F}$$

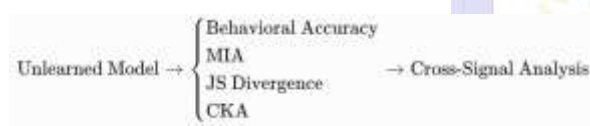
where (X) and (Y) are the centered feature matrices from the two models, and ($\|\cdot\|_F$) is the Frobenius norm. Like JS divergence, CKA runs separately for forget-test and retain-test samples; higher values mean the unlearned model's representations are closer to the retrained reference's. CKA is computed via a memory-safe, feature-space implementation, avoiding unnecessarily large kernel matrices. CKA adds a representation-level view that behavioral and output-level evidence can't offer, checking whether internal representations shift in ways prediction accuracy or output distributions don't capture.

5) Complementary Role of the Evidence Types

Each of the four evidence types picks up on a different observable slice of the unlearned model:

Evidence Type	Primary Perspective	Main Observation
Behavioral accuracy	Predictive behavior	Forget and retain classification performance
MIA	Membership-related behavior	Separability of forget-set members and non-members
JS divergence	Output behavior	Similarity to the condition-specific retrained reference
CKA	Internal representation	Similarity of penultimate-layer representations

The whole verification pipeline can be summarized like this:



The specific implementation and evaluation settings for all of this

are laid out in Section 5.

I. Cross-Signal Consistency Analysis

All four evidence types are examined together to see whether they agree within the same run, or pull in different directions. The goal isn't a universal cutoff for "successful" forgetting — just whether these four signals under the same condition tell a consistent story. For each unlearning run, four forgetting-oriented signals are considered side by side:

- behavioral forget-set accuracy,
- MIA ROC-AUC,
- forget-set JS divergence, and
- forget-set CKA.

These measures live on different scales, so raw numbers are never combined into one score. Instead, each signal is read relative to its own condition-level reference: the within-condition median across the three seed repetitions. That lets the three runs sharing a method-condition combination be compared without an arbitrary universal pass/fail line. Each run's four directional readings are then compared against one another. Based on how those four directions line up, every run falls into one of three buckets:

1. **Mixed evidence** — the four signals point in different directions, with no shared directional story.

2. **Tie/neutral** — the comparison doesn't produce a clear directional split at all.

3. **Unanimous directional evidence** — all four signals agree on the same direction.

The results reported here come from the **direction-based procedure (Procedure 14a)**. An earlier threshold-based version was tried first but treated as preliminary and dropped in favor of this one for the final reported results.

Across all 24 unlearning runs, this is how that shook out:

Consistency Pattern	Runs	Percentage
Mixed evidence	14	58.33%
Tie/neutral	10	41.67%
Unanimous directional evidence	0	0%

So not a single one of the 24 runs achieved unanimous agreement across all four signals. Most runs — **14 out of 24, or 58.33%** — landed in mixed evidence, and the remaining **10 (41.67%)** came out tie/neutral. Splitting this by method: GA_FT came in at **6 mixed and 6 tie/neutral**, while SalUn landed at **8 mixed and 4 tie/neutral**. These numbers are purely descriptive here — meant to characterize the pattern of evidence, not to declare one method better or worse than the other. At the method-condition level, all three seed runs for every combination were classified the same way; none reached unanimous agreement across all runs and signals. This offers an empirical snapshot of how established verification signals behave when applied to the same unlearned models — separating cases where evidence roughly agrees from cases where signals tell markedly different stories.

To sum up, the cross-signal analysis runs like this:

Four Established Signals >Condition-Relative Directional >Comparison >Agreement Classification >Consistency Characterization

This is where the study's central research question gets put into practice — checking how a fixed set of verification evidence responds as forgetting conditions change.

J. Reproducibility

To keep experiments reproducible, several elements were held fixed across all runs: dataset preprocessing, model architecture, forgetting conditions, the verification protocol, and random seeds. Every method-condition combination was run with the same three seeds — 42, 123, and 2026 — giving three independent repetitions per pairing. Every unlearning run started from the same original checkpoint rather than chaining runs together, since sequential unlearning would let earlier runs bleed into later comparisons; each condition needed to be judged on its own terms. The four forgetting conditions — forget classes, retain sets, difficulty values — were locked in before any unlearning began and never revisited based on results, avoiding letting outcomes shape the experimental design. Image preprocessing and normalization stayed identical across the original model, unlearning experiments, retrained reference models, and evaluation. The architecture also didn't change between original, unlearned, and reference models. Verification was handled identically for every run: each unlearned model went through the same four checks and was compared against the retrained reference built for its specific condition.

In total, this design produced 24 unlearning runs:

2 unlearning methods × 4 forgetting conditions × 3 random seeds = 24 runs

Four retrained reference models were trained, one per condition, with checkpoints, metadata, condition definitions, and verification outputs kept as artifacts. These artifacts — the original checkpoint, all unlearned checkpoints,

retrained reference models, and the results from every evaluation stage — document the full configuration and output behind everything reported here.

Fixed original checkpoint → frozen forgetting conditions → fixed random seeds → identical verification protocol → repeated method-condition runs → preserved experimental artifacts.

Locking these pieces down ahead of time limits variation that could creep in unintentionally, so reported comparisons reflect the original design rather than later adjustments.

IV. Experimental Setup

A. Hardware and Software Environment

All experiments ran in a GPU-enabled Python environment, with a Tesla T4 GPU handling model training, unlearning, retraining, and verification. The pipeline was built in Python with PyTorch as the primary deep-learning framework. The table below lists the main software versions used:

Component	Version / Configuration
GPU	NVIDIA Tesla T4
Deep-learning framework	PyTorch 2.11.0 + CUDA 12.8
Torchvision	0.26.0 + CUDA 12.8
NumPy	2.1.3
Pandas	2.2.3
Matplotlib	3.10.0
Scikit-learn	1.6.1
Dataset	CIFAR-10

PyTorch and Torchvision handled dataset processing, model construction, training, unlearning, and feature extraction; scikit-learn handled statistical and evaluation procedures; NumPy and Pandas handled numerical processing and aggregation; Matplotlib produced the figures. The same computational and software setup was kept across all major stages to avoid environment-driven variation: the original model, unlearning runs, retrained models, and verification analyses all ran within the same project environment and implementation. Experiments were organized into a structured project directory holding checkpoints, condition definitions, metadata, verification outputs, cross-verification analyses, and result summaries, letting each stage's output be retained and drawn on for the paper's analysis.

B. Original Model Training

The original CIFAR-10 classification model was trained using the customized ResNet-18 architecture described in Section 4.3. The original training data was split into **45,000 training samples and 5,000 validation samples**, with the validation set used to track performance during training and pick the best checkpoint. Training ran for up to **100 epochs** using stochastic gradient descent, with a batch size of **128**, an initial learning rate of **0.1**, momentum of **0.9**, and weight decay of (5^{-4}) . A cosine-annealing schedule adjusted the learning rate over the course of training. The best validation result came at **epoch 78**, reaching **88.02%** validation accuracy. On the full CIFAR-10 test set, the resulting model achieved **87.28%** test accuracy. This checkpoint was fixed and used as the starting point for every unlearning experiment — none of the 24 runs started from a separately trained model. The original model plays two

roles: the shared starting point for GA_FT and SalUn, and the source of predictions feeding the forgetting-difficulty values in Section 4.5. It therefore had to be established before conditions were defined or outcomes evaluated.

C. Unlearning Configuration

Both methods were applied with fixed configurations across all conditions and seeds, so any changes in verification evidence could be attributed to the forgetting conditions rather than shifting hyperparameters.

1) GA_FT Configuration:- GA_FT ran as two sequential stages — a gradient-ascent forgetting stage followed by retain-set fine-tuning. The forgetting stage ran for 3 epochs, followed by 3 epochs of retain-set fine-tuning. SGD was used with a learning rate of 0.001, momentum of 0.9, and weight decay of (5^{-4}) , plus gradient clipping at a max norm of 1.0 to keep parameter updates from spiking. This same setup applied across all four forgetting conditions and all three seeds.

2) SalUn Configuration:- SalUn was likewise a two-stage procedure. The first stage worked on the designated forget set with randomized labels, combined with the saliency-guided forgetting mechanism, and ran for 3 epochs. The second stage fine-tuned on the retain set for 3 epochs to restore retained-task performance. A learning rate of 0.001 was used, along with a 50% saliency ratio and gradient clipping at a maximum norm of 1.0. Other optimization settings were retained as implemented in the experimental notebook and are not separately reported here. A 50% saliency ratio determined which subset of parameters the saliency-guided forgetting procedure affected, and gradient clipping was again capped at a max norm of 1.0.

3) Summary of Unlearning Configurations

Parameter	GA_FT	SalUn
Forgetting-stage epochs	3	3
Retain fine-tuning epochs	3	3
Learning rate	0.001	0.001
Momentum	0.9	—
Weight decay	(5^{-4}) –	-
Gradient clipping	1.0	1.0
Saliency ratio	—	50%
Forget-stage objective	Gradient ascent	Randomized-label saliency-guided forgetting
Retain-stage objective	Retain-set fine-tuning	Retain-set fine-tuning

This exact configuration was applied independently to every method-condition-seed combination, giving a total of:

- $(2 \times 4) \times 3 = 8 \times 3 = 24$
- $2 \times (4 \times 3) = 2 \times 12 = 24$

runs — spanning the two unlearning methods, four forgetting conditions, and three random seeds.

D. Forgetting Conditions

The conceptual conditions from Section 4.4 were turned into four fixed, class-based forgetting scenarios, varying two factors: forget-set size and forgetting difficulty. Small conditions involved a single forgotten class; large conditions involved two. Class selections came from the difficulty-construction procedure in Section 4.5 and were locked in before unlearning began.

The final conditions were:

Condition	Forget Classes	Approx. Size	Difficulty
Small-Easy	Automobile	10%	4.80%
Small-Hard	Cat	10%	26.00%
Large-Easy	Automobile + Truck	20%	3.15%
Large-Hard	Bird + Cat	20%	18.25%

The corresponding data partitions were:

Condition	Forget Train	Retain Train	Forget Test	Retain Test
Small-Easy	4,488	40,512	1,000	9,000
Small-Hard	4,529	40,471	1,000	9,000
Large-Easy	8,987	36,013	2,000	8,000
Large-Hard	8,997	36,003	2,000	8,000

The same condition definitions were used for GA_FT, SalUn, and the retrained reference models, untouched once results started coming in. Together, these four conditions make up the experimental matrix behind the whole study:

$2 \text{ methods} \times 4 \text{ conditions} \times 3 \text{ seeds} = 24 \text{ total runs}$ This setup gives controlled variation in both how much information the model is asked to forget and how hard it is to separate forget classes from retained ones.

E. Random Seeds

To capture how results vary across repeated runs, three fixed random seeds were used: 42, 123, and 2026. Every method-condition combination was run independently under all three. This gives an experimental structure of:

$2 \text{ methods} \times 4 \text{ conditions} \times 3 \text{ seeds} = 24 \text{ unlearning runs}$ The same seeds were applied to both methods across all four conditions, so each pairing ended up with exactly three runs — enough to summarize verification-measurement variability using mean and standard deviation. The seed configuration was locked in before the final experiments ran and wasn't adjusted afterward. All 24 planned runs completed successfully. Repeating seeds matters because the study looks at how consistent verification responses are across multiple runs, rather than treating one run as representative; seed variation is carried through into the analysis itself.

F. Evaluation Protocol

Once each run finished, the model went through the fixed protocol from Section 4.8, applied identically across all 24 models, in four stages.

1) Behavioral Evaluation:- Each unlearned model was tested on the forget-test and retain-test subsets for its condition, recording forget, retain, and overall accuracy — separating changes tied to forgetting from changes in retained-task performance.

2) Membership Inference Evaluation:- For MIA, forgotten-class training samples served as members and matching test samples as non-members. Recorded per run: member/non-member counts, mean losses, and MIA ROC-AUC. The attack score used negative per-sample cross-entropy loss, with ROC-AUC measuring separability. This protocol ran across every method-condition-seed combination; all 24 runs were evaluated, with member/non-member counts varying as expected by condition.

3) Output-Similarity Evaluation:- Each unlearned model's predictive distributions were compared against its condition-specific retrained reference, with JS divergence calculated separately for forget-test and retain-test samples and summarized across seeds via mean and standard deviation — an output-level comparison holding the forgetting condition constant.

4) Representation-Similarity Evaluation

Representation-level comparison used linear CKA on the 512-dimensional penultimate-layer features from Section 4.3, collected on forget-test and retain-test samples and compared against the matching retrained reference. Resulting forget- and retain-CKA values were recorded for every run; evaluation artifacts confirm all 24 unlearned and all four reference checkpoints were available for this analysis.

5) Aggregation Across Random Seeds

For each method-condition combination, the three seed-specific measurements were summarized with:

and

$$\text{Mean} = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\text{SD} = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

where $n=3$ is the number of seed runs. These mean $\pm$ standard deviation values are the primary summary statistics reported in the Results section.



No verification measure was dropped or altered based on outcome — all four evidence types were evaluated for every run and analyzed together.

V. Results

A. Original Model Performance

Before examining the unlearning procedures, the original ResNet-18 was evaluated on the full CIFAR-10 test set and on each condition's forget/retain subsets. It reached 87.28% overall test accuracy, the baseline for the rest of the experiments. Condition-specific results appear in Table 3.

Table 3. Condition-Specific Performance of the Original Model

Condition	Forget Accuracy (%)	Retain Accuracy (%)	Overall Accuracy (%)
Small-Easy	95.20	86.40	87.28
Small-Hard	74.00	88.76	87.28
Large-Easy	93.90	85.62	87.28
Large-Hard	78.25	89.54	87.28

The original model's forget-class accuracy varied widely depending on the target: 95.20% on automobile versus 74.00% on cat in the small conditions; 93.90% on automobile+truck versus 78.25% on bird+cat in the large conditions. This lines up with how difficulty was constructed in Section 4.5 — harder conditions were deliberately built around classes the original model already confused with retained ones. Overall accuracy stays fixed at 87.28% across all four rows, since each condition is simply a different slice of the same test-set predictions rather than a separate model run. This baseline serves as context for interpreting the forget/retain subsets, not as four separate measurements of overall performance. With that established, the original model gives us a shared reference point for tracking what changes once unlearning is applied.

B. Unlearning Results

The two methods didn't respond the same way across the four conditions. Seed-level results are reported as mean $\pm$ standard deviation across the three seeds.

1) GA_FT:- GA_FT's forget-set accuracy swung considerably depending on the condition.

Table 6. Behavioral Performance of GA_FT Across Forgetting Conditions

Condition	Forget Accuracy (%)	Retain Accuracy (%)	Overall Accuracy (%)
Small-Easy	92.90 $\pm$ 0.17	86.46 $\pm$ 0.11	87.11 $\pm$ 0.09
Small-Hard	67.37 $\pm$ 0.74	89.13 $\pm$ 0.13	86.96 $\pm$ 0.04
Large-Easy	36.97 $\pm$ 0.97	85.44 $\pm$ 0.10	75.74 $\pm$ 0.27
Large-Hard	16.60 $\pm$ 0.05	91.86 $\pm$ 0.09	76.81 $\pm$ 0.07

Forget accuracy spanned 16.60% to 92.90% across the four conditions, peaking in small-easy and bottoming out in large-hard. Retain accuracy stayed comparatively steady, between 85.44% and 91.86% — GA_FT's largest behavioral shifts showed up mainly on the forget classes, not spread evenly across forget and retain. Overall accuracy also moved around, from 75.74% to 87.11%, largely tracking the sizable swings in forget-set performance seen in the two large conditions.

2) SalUn:- SalUn told a different story:

Table 7. Behavioral Performance of SalUn Across Forgetting Conditions

Condition	Forget Accuracy (%)	Retain Accuracy (%)	Overall Accuracy (%)
Small-Easy	89.70 $\pm$ 0.79	86.50 $\pm$ 0.05	86.82 $\pm$ 0.08
Small-Hard	80.47 $\pm$ 0.29	87.46 $\pm$ 0.06	86.76 $\pm$ 0.05
Large-Easy	58.80 $\pm$ 0.74	85.74 $\pm$ 0.08	80.35 $\pm$ 0.11
Large-Hard	78.15 $\pm$ 0.48	89.40 $\pm$ 0.08	87.15 $\pm$ 0.05

Its forget accuracy ranged from 58.80% to 89.70%. But unlike GA_FT, the weakest point here was large-easy, not large-hard. Retain accuracy stayed within a narrower band of 85.74% to 89.40%, and overall accuracy landed between 80.35% and 87.15%. Across both methods, seed-level standard deviations were generally small (e.g., GA_FT's large-hard forget accuracy: 16.60 $\pm$ 0.05%; SalUn's: 78.15 $\pm$ 0.48%) — differences between conditions far exceeded seed-to-seed noise.

C. Behavioral Verification

Overall, the behavioral results show that forget-set accuracy depended heavily on the condition for both methods. For GA_FT, the gap between its highest and lowest forget accuracy across conditions was:

$$92.90 - 16.60 = 76.30 \text{ percentage points}$$

For SalUn, that same gap was:

$$89.70 - 58.80 = 30.90 \text{ percentage points}$$

So GA_FT's behavioral response swung across a much wider range than SalUn's. GA_FT's response to size and difficulty: moving from easy to hard dropped forget accuracy by 25.53 points for the small forget set and 20.37 points for the large one; increasing forget-set size dropped it by 55.93 points under easy conditions and 50.77 under hard. SalUn's pattern differed: increasing forget-set size cost 30.90 points under easy conditions but only 2.32 under hard; the easy-to-hard contrast flipped sign by size — -9.23 points for small, +19.35 for large. These are descriptive comparisons drawn from four conditions per method, characterizing the observed pattern rather than supporting broader inferential claims. Even so, the behavioral results signal that an established method's response can shift substantially with the forgetting condition — and the two methods clearly don't follow the same condition-response pattern. Behavioral accuracy is just one piece of the verification picture. Whether this condition-dependence also shows up in privacy-, output-, and representation-level evidence is examined next.

D. Membership Inference Attack (MIA)

MIA gives a privacy-focused angle: members were forgotten-class training samples, non-members the matching test samples. The attack score used negative per-sample cross-entropy loss, with ROC-AUC measuring group distinguishability. GA_FT's MIA ROC-AUC values were 0.6038 ± 0.0002 , 0.8968 ± 0.0006 , 0.5099 ± 0.0006 , and 0.7542 ± 0.0032 for small-easy, small-hard, large-easy, and large-hard. SalUn's were 0.6731 ± 0.0047 , 0.9162 ± 0.0004 , 0.6634 ± 0.0014 , and 0.8954 ± 0.0003 . Across all eight method-condition pairings, ROC-AUC ranged from 0.5099 (GA_FT, large-easy) to 0.9162 (SalUn, small-hard). Member/non-member losses varied widely too: GA_FT under large-easy gave a mean member loss of 6.8102 against 6.7606 (ROC-AUC 0.5099), while under small-hard the member loss dropped to 0.0019 against 1.0744 (ROC-AUC 0.8968). For SalUn: small-easy gave member/non-member losses of 0.1402/0.4385; small-hard 0.0006/0.5863; large-easy 0.6888/1.0356; large-hard 0.0095/0.6986. MIA responded differently depending on the condition, and importantly, that response doesn't simply mirror the behavioral forget-accuracy pattern — part of why MIA is treated as complementary evidence rather than a restatement of behavioral forgetting. An ROC-AUC closer to 0.5 means the groups were harder to distinguish; higher values mean more distinguishable groups. MIA results should be read as one piece of verification evidence, not standalone proof either way.

E. Output Similarity Using Jensen-Shannon Divergence

For output-level similarity, each unlearned model's predictive distributions were compared against its condition-specific retrained reference, with JS divergence calculated separately for forget and retain samples. GA_FT's forget-set JS divergence: 0.6192 ± 0.0011 , 0.4115 ± 0.0037 , 0.5640 ± 0.0070 , and 0.3020 ± 0.0025 for small-easy, small-hard, large-easy, and large-hard.

Retain-set JS values, by contrast, were much smaller across the board:

- small-easy: 0.0447 ± 0.0001
- small-hard: 0.0417 ± 0.0004

- large-easy: 0.0522 ± 0.0002
- large-hard: 0.0367 ± 0.0004

For SalUn, forget-set JS divergence was:

- small-easy: 0.4968 ± 0.0179
- small-hard: 0.4975 ± 0.0008
- large-easy: 0.4369 ± 0.0043
- large-hard: 0.4971 ± 0.0012

And retain-set values were:

- small-easy: 0.0452 ± 0.0002
- small-hard: 0.0534 ± 0.0002
- large-easy: 0.0530 ± 0.0001
- large-hard: 0.0532 ± 0.0004

One pattern holds across both methods: forget-set JS divergence was consistently far higher than retain-set JS divergence in every condition — unlearned models' outputs diverged more from the reference on forgotten samples than retained ones. The size of that gap varied by condition: GA_FT's forget-set JS ranged widely (0.3020–0.6192) while SalUn's stayed tighter (0.4369–0.4975) — GA_FT showed more condition-level variation here. Retain-set values stayed comparatively steady for both methods (0.0367–0.0534) — retained-sample output behavior stayed close to the reference regardless of condition. The JS results add a distinct output-level view: the gap between forgotten and retained samples' similarity to the reference depends on both the condition and the method.

F. Representation Similarity Using CKA

For representation-level comparison, linear CKA was computed on the 512-dimensional penultimate-layer features, comparing each unlearned model against its condition-specific retrained reference for forget and retain samples.

For GA_FT, forget-set CKA values were:

- small-easy: 0.1065 ± 0.0031
- small-hard: 0.2948 ± 0.0022
- large-easy: 0.0563 ± 0.0022
- large-hard: 0.4670 ± 0.0055

Retain-set CKA values were:

- small-easy: 0.9188 ± 0.0004
- small-hard: 0.9347 ± 0.0009
- large-easy: 0.8998 ± 0.0002
- large-hard: 0.9251 ± 0.0003

For SalUn, forget-set CKA came out to:

- small-easy: 0.1453 ± 0.0097
- small-hard: 0.1981 ± 0.0023
- large-easy: 0.1943 ± 0.0046
- large-hard: 0.2955 ± 0.0024

With retain-set values of:

- small-easy: 0.9153 ± 0.0001

- small-hard: 0.9295 ± 0.0010
- large-easy: 0.8994 ± 0.0006
- large-hard: 0.9200 ± 0.0011

A clear split shows up for both methods: retain-set CKA stayed high across every condition, while forget-set CKA came out noticeably lower throughout. GA_FT's forget-set CKA ranged fairly widely, from 0.0563 to 0.4670, while SalUn's was narrower, from 0.1453 to 0.2955. High retain-set CKA shows retained samples kept representations close to the reference; lower forget-set values show larger representational differences on forgotten samples — though not constant: GA_FT dropped to 0.0563 under large-easy but rose to 0.4670 under large-hard, showing representation-level evidence also shifts with the condition. So CKA contributes its own representation-level angle here, one that doesn't simply echo behavioral accuracy, MIA, or output similarity.

G. Comparison with Condition-Specific Retrained References

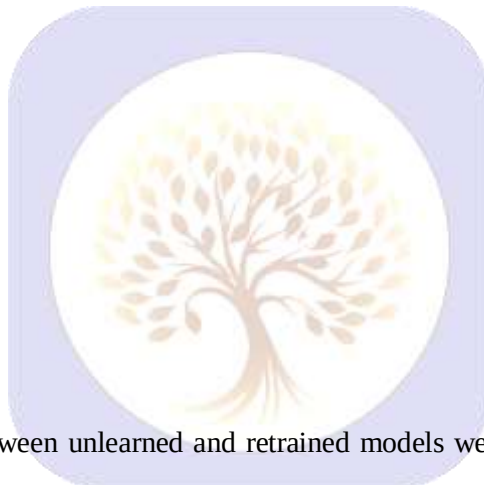
The retrained models served as condition-specific reference points. Since their training data excluded the relevant forget classes entirely, they landed at 0.00% forget-set accuracy in all four conditions.

Their retain-set accuracy was:

- small-easy: 86.04%
- small-hard: 89.53%
- large-easy: 84.44%
- large-hard: 90.75%

And their overall accuracy:

- small-easy: 77.44%
- small-hard: 80.58%
- large-easy: 67.55%
- large-hard: 72.60%



From there, the behavioral gaps between unlearned and retrained models were examined. For GA_FT, the retain-accuracy gaps were:

- small-easy: 0.4185
- small-hard: -0.4000
- large-easy: 1.0000
- large-hard: 1.1083

And the overall accuracy gaps were:

- 9.6667
- 6.3767
- 8.1933
- 4.2067

For SalUn, retain-accuracy gaps were:

- small-easy: 0.4556
- small-hard: -2.0741
- large-easy: 1.3000
- large-hard: -1.3500

With overall gaps of:

- 9.3800
- 6.1800
- 12.8000
- 14.5500

Since the references sit at 0% forget accuracy everywhere, the forget-accuracy gap simply mirrors the unlearned model's own forget accuracy already covered in Section 6.3. The retain and overall gaps add more: they show how unlearned and retrained models relate beyond the forget classes, matching the pattern already visible in the similarity measures — low retain-set JS divergence and high retain-set CKA relative to forget-set counterparts. The retrained models are best thought of as condition-specific reference points, not universal ground truth — their job is to give each scenario a consistent yardstick.

H. Cross-Signal Verification Consistency

To check whether verification signals agreed within a given run, four forgetting-oriented signals were examined together: forget-set accuracy, MIA ROC-AUC, forget-set JS divergence, and forget-set CKA. The consistency check used the within-condition median across the three seeds as the directional reference, sorting each run into mixed evidence, tie/neutral, or unanimous agreement.

Across all 24 runs:

- Mixed evidence: 14/24 (58.33%)
- Tie/neutral: 10/24 (41.67%)
- Unanimous forgetting: 0/24
- Unanimous non-forgetting: 0/24

So not a single one of the 24 runs saw all four signals point in the same direction. Breaking it down by method, GA_FT had 6 mixed runs and 6 tie/neutral runs, while SalUn had 8 mixed and 4 tie/neutral.

At the condition level, it was a similar mix rather than clean agreement. GA_FT came out as:

- large-easy: 2 mixed / 1 tie
- large-hard: 1 mixed / 2 tie
- small-easy: 1 mixed / 2 tie
- small-hard: 2 mixed / 1 tie

SalUn landed at 2 mixed / 1 tie in every single condition. The four evidence types didn't reliably move together across runs. Mixed evidence isn't proof unlearning failed, and tie/neutral isn't proof it succeeded — these categories just describe directional agreement among the chosen signals. That none of the 24 runs hit unanimous agreement is itself an empirical signal that these measures can respond differently even on the same unlearned model.

I. Effect of Forgetting Conditions on Verification Evidence

The last analysis looked at how signals shifted across the four conditions by forget-set size and difficulty. With only four conditions per method, these are descriptive contrasts, not statistically inferential ones. For GA_FT, forget accuracy spanned 16.60% to 92.90% (76.30-point range); MIA 0.5099–0.8968; JS divergence 0.3020–0.6192; CKA 0.0563–0.4670. For SalUn, forget accuracy spanned 58.80% to 89.70% (30.90-point range); MIA 0.6634–0.9162; JS divergence 0.4369–0.4975; CKA 0.1453–0.2955. Looking at the size and difficulty contrasts fills in the picture further. For GA_FT, the hard-minus-easy contrast in forget accuracy was –25.53 points (small) and –20.37 points

(large); the large-minus-small contrast was -55.93 points (easy) and -50.77 points (hard). For SalUn: -9.23 points (small, hard-minus-easy), $+19.35$ points (large, hard-minus-easy), -30.90 points (easy, large-minus-small), and -2.32 points (hard, large-minus-small). The other evidence types showed their own distinct condition-level patterns.

For GA_FT:

Evidence	Difficulty: Small	Difficulty: Large	Size: Easy	Size: Hard	Range
Forget Accuracy	-25.53	-20.37	-55.93	-50.77	76.30
MIA	$+0.2930$	$+0.2443$	-0.0939	-0.1426	0.3869
JS	-0.2077	-0.2620	-0.0552	-0.1095	0.3172
CKA	$+0.1883$	$+0.4107$	-0.0502	$+0.1723$	0.4107

For SalUn:

Evidence	Difficulty: Small	Difficulty: Large	Size: Easy	Size: Hard	Range
Forget Accuracy	-9.23	$+19.35$	-30.90	-2.32	30.90
MIA	$+0.2431$	$+0.2320$	-0.0097	-0.0209	0.2528
JS	$+0.0007$	$+0.0602$	-0.0598	-0.0003	0.0605
CKA	$+0.0529$	$+0.1013$	$+0.0490$	$+0.0974$	0.1503

These contrasts show there's no single, uniform way verification evidence reacts to changing conditions — the size and even direction of the shift depends on both the evidence type and the method. This fits the cross-signal analysis: 14 of 24 runs (58.33%) had mixed directional evidence and 10 (41.67%) were tie/neutral, with no run showing full agreement. Established verification signals can respond differently as forget-set size and difficulty change, even with everything else about the setup fixed — setting up the discussion in the next section.

VI. Discussion

A. Behavioral Evidence

The behavioral evidence makes one thing clear: forgetting can't be interpreted on its own terms, because it depends heavily on what the forget set actually looks like. The same procedure behaved quite differently depending on which data it was asked to forget, and that matters because behavioral accuracy is usually the first thing people check — yet how you read it depends on how the original model treated that forget set to begin with. Because difficulty here was built directly from the original model's pre-existing confusion, the behavioral response after unlearning can be weighed against that known starting gap. What this tells us is that apparent forgetting isn't just about the unlearning procedure itself; it's also shaped by what kind of information is being asked to disappear. Behavioral evidence, though, only tells us about prediction behavior. A drop in accuracy doesn't reveal what happened internally, or whether the model has actually become indistinguishable from one trained without that data — a distinction that matters here, since the other signals didn't necessarily track the same pattern. So behavioral accuracy is a useful signal, but it means a lot more when read alongside privacy-, output-, and representation-level evidence, which is exactly the premise behind treating it as one piece of a multi-dimensional picture rather than the whole story.

B. Privacy Evidence

MIA offers a distinctly different angle from behavioral accuracy: it asks whether forgotten samples can still be told apart from non-members under a defined attack, which is a fundamentally different property from classification accuracy altogether. MIA shifted across the same conditions where behavioral responses shifted, but not in the same

way; membership distinguishability simply doesn't move in lockstep with behavioral forgetting. A model can look forgotten behaviorally while looking quite different under a membership lens. This lines up with the literature's caution against relying on membership inference as an unlearning benchmark: MIA results hinge on attack design, on how members and non-members are defined, on the scoring function, and on the data itself. These particular ROC-AUC numbers reflect this specific setup, not some universal verdict on erasure. Still, the privacy angle adds something behavioral evaluation can't offer on its own: using forgotten training samples as members and their test counterparts as non-members checks whether unlearning actually shifts that distinguishability. The fact that it didn't simply echo the behavioral pattern reinforces the value of checking multiple dimensions at once. The MIA findings back up a core idea running through this paper: different verification signals tell you different things about the same model, and each one needs to be interpreted in light of the protocol that produced it.

C. Output and Representation Evidence

JS and CKA offer two complementary windows into how unlearned models relate to their references: JS looks at differences in predictive output, while CKA looks at internal-representation similarity — the two operate at entirely different levels of the model. Both showed a clear split between forget and retain samples: retained samples stayed much closer to the reference on both fronts, while forgotten samples showed more divergence and lower representation similarity. But the two didn't move together across conditions, which is worth dwelling on, since output similarity and representation similarity aren't interchangeable. Two models can produce near-identical predictions while differing quite a bit internally, and representation differences don't automatically show up in predicted labels or probabilities. This lines up with recent representation-focused work arguing that output behavior alone doesn't reveal everything left in a model's internals — the “Illusion of Unlearning” paper makes this point well. This study isn't extending that line of work or proposing a new metric; it simply applies CKA as an established representation-level signal within a fixed protocol. Together, JS and CKA make the case for complementary evidence: JS captures prediction-level behavior, CKA captures representation-space similarity, and the fact that they respond differently across conditions shows that no single level of analysis fully captures an unlearned model on its own.

D. Condition-Dependent Verification

The single biggest takeaway is that verification responses are condition-dependent. The conditions here were built by varying forget-set size and difficulty while the verification protocol stayed fixed, and the shifts seen across behavioral, MIA, JS, and CKA measurements show that each signal's response depends partly on the scenario the forgetting was carried out under. The same logic holds for every signal: MIA depends on the member/non-member relationship, JS and CKA hinge on comparison against the matching reference, and all four are shaped by whatever data is being evaluated. Verification evidence needs to be read together with the conditions that generated it, not in isolation from them. This condition-dependence backs up the paper's narrower point: the goal here was never to introduce a new metric or “discover” flaws, but to take a controlled, empirical look at how established verification signals behave once forgetting conditions are systematically varied.

E. Cross-Signal Heterogeneity

The most important result to come out of the cross-signal analysis is that the four signals didn't consistently agree on direction within a given run. That's not entirely surprising once you consider that each one is tracking a different

property: behavioral accuracy tracks prediction, MIA tracks membership distinguishability, JS tracks output-distribution difference against a reference, and CKA tracks internal-representation similarity.

F. Comparison with Existing Literature

Prior work has already shown that unlearning can be assessed through multiple kinds of evidence, and that any single approach has its limits, with conclusions shifting depending on criterion, dataset, or setup. This study follows that same multi-dimensional mindset rather than introducing yet another standalone metric. Existing work on what makes unlearning difficult — the “What Makes Unlearning Hard” line of research — shows that forget-set characteristics shape unlearning behavior directly, which justifies treating difficulty as a controlled factor here. What’s different in this study is the focus on how verification evidence itself shifts as conditions change. A growing body of work questions whether output-level behavior alone is enough to show that information has actually been removed; the “Illusion of Unlearning” paper makes a strong case for looking past outputs alone. Using CKA here fits squarely into that direction, applied as an established tool rather than offered up as a new contribution. The 2026 survey on machine unlearning verification pushes back on treating verification and evaluation as the same thing, flagging inconsistencies across criteria. What’s found here fits squarely into that concern: this controlled experiment also turned up heterogeneous responses across established evidence types. Throughout, the verification setup was held constant while forget-set size and difficulty were deliberately varied across four conditions, with two methods tested under the same protocol across multiple seeds. That design is what lets us connect changes in verification evidence back to controlled changes in the forgetting scenario.

G. Research Implications

A few implications follow from this for how machine unlearning gets evaluated and verified going forward. This work isn’t proposing a new verification metric or a universal decision rule. What it supports instead is a more condition-aware, multi-evidence way of reading established verification signals, along with a more cautious stance on forgetting claims whenever the evidence types don’t all point the same way.

VII. Limitations

Several limitations are worth flagging here, since they mark out the boundaries within which these findings should be read. The experimental scope is narrow: a single dataset and a single architecture, CIFAR-10 and a custom ResNet-18. That narrowness buys a controlled setting, but it also means the patterns observed here can’t simply be assumed to hold for other datasets, architectures, data types, or larger models. Only two established methods were tested, GA_FT and SalUn. They’re usefully contrasting procedures, but nowhere near the full range of unlearning approaches out there, so these method-specific patterns shouldn’t be taken as representative of unlearning algorithms in general. The forgetting conditions also cover a deliberately narrow space: one-class versus two-class forget sets and predefined easy/hard difficulty give four controlled conditions, systematic in design but leaving out other dataset percentages, class combinations, instance-level forgetting, and other difficulty definitions. The difficulty definition itself is specific to this setup, based on how often the original model misclassified forget-set samples as retained classes. It’s workable and reproducible here, but not universal — other datasets, models, or metrics could well sort easy and hard differently. The retrained models, too, aren’t meant as universal ground truth. They’re useful condition-specific references for behavioral, output-, and representation-level comparisons, but retraining from scratch doesn’t formally prove the

original information is actually gone. Any gap between an unlearned model and its reference counts as comparative evidence, not certification.

The MIA results are similarly tied to this specific protocol: forgotten-class training samples as members, matching test samples as non-members, and negative per-sample cross-entropy loss as the attack score. A different attack design, population, or scoring function could well produce different numbers. The cross-signal consistency analysis is descriptive by design, too. It doesn't establish statistical significance for condition effects, and with only four conditions per method, there's limited basis for drawing broader inferences. These patterns are empirical observations specific to this setting, not general laws. Nor is the verification evidence used here exhaustive. Behavioral accuracy, MIA, JS divergence, and CKA each add something useful, but the broader literature also includes other privacy attacks, robustness tests, statistical criteria, and formal or third-party verification methods. What's characterized here is how this particular set of evidence types behaves, not verification as a whole. None of this amounts to a formal guarantee that information has been completely removed, either. The goal was to characterize how established evidence responds, and how consistent it is, under controlled conditions — not to certify that every trace of forgotten data is gone. A guarantee like that would need additional assumptions and, most likely, formal or cryptographic methods well beyond this study's scope. Taken together, these limitations mark out what this contribution actually covers: a controlled, empirical look at how verification responses hold up, or don't, across varying forgetting conditions. Extending beyond the specific dataset, architecture, methods, conditions, and protocols used here would take further work.

VIII. Conclusion and Future Work

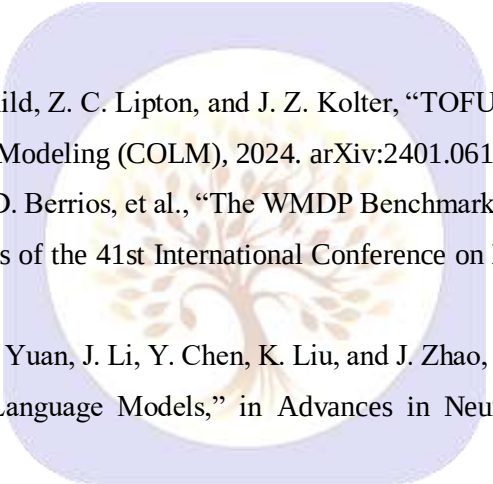
A. Conclusion

This study set out to test how consistent established machine-unlearning verification evidence remains when forgetting conditions are varied. Rather than proposing a new algorithm or metric, it kept the verification framework fixed and deliberately varied forget-set size and forgetting difficulty instead. Using CIFAR-10 and a custom ResNet-18, two methods, GA_FT and SalUn, were tested across four predefined conditions and three seeds, for 24 runs in total. Four evidence types were examined along the way: behavioral performance, membership inference, JS output similarity, and CKA representation similarity, with condition-specific retrained models serving as reference points. Verification responses turned out to be both condition-dependent and inconsistent across evidence types. Forget accuracy ranged from 16.60% to 92.90% for GA_FT and from 58.80% to 89.70% for SalUn, and MIA, JS divergence, and CKA all showed that same condition-dependence. In other words, the verification evidence you end up with can shift substantially depending on the forgetting condition applied. The cross-signal analysis added another layer on top of that: across 24 runs, 14 (58.33%) produced mixed directional evidence and 10 (41.67%) came out tie/neutral, with not a single run showing all four signals in agreement. That doesn't mean a method definitively “worked” or “failed” — just that established forms of evidence can genuinely disagree on the very same run. So the answer to the research question is this: even with the verification protocol held constant, what it reports still shifts with the forgetting condition, particularly forget-set size and difficulty. The evidence is not a uniform, stable indicator across conditions; results need to be read in light of both the scenario they came from and what each piece of evidence actually measures. This study's contribution, then, is an empirical picture of how consistent, or inconsistent, verification evidence really is under controlled changes to the forgetting setup — making the case for reading multiple complementary forms of evidence with the condition in mind, rather than treating any one signal as a universal stamp of success.

B. Future Work

There's plenty of room to build on this work. One direction would be widening the range of conditions tested — more forget-set sizes, class combinations, instance-level forgetting, and difficulty definitions — to see whether these condition-dependent patterns hold up more broadly. Bringing more unlearning methods into the analysis would show how the relationship between strategy and verification-response consistency plays out across a wider range of algorithmic behavior. Extending the work to other datasets and architectures would help clarify whether these findings are specific to this particular setting or show up more broadly. There's also room to bring in more verification evidence — other privacy attacks, robustness checks, statistical criteria, and formal or third-party methods — for a fuller picture of verification consistency. Digging into why signals disagree in the first place is a natural next step: this study shows that mixed and neutral patterns do occur, but it doesn't pin down which properties of the forget set, model, method, or protocol drive them the most. There's real value, too, in pushing toward more formal verification — pairing this kind of empirical, multi-evidence analysis with formal, statistical, or cryptographic methods to close the gap between empirical signs of forgetting and stronger claims about information removal. Future work should stretch this controlled framework across more conditions, methods, datasets, architectures, and verification approaches, all while keeping empirical evidence clearly distinct from formal guarantees that information has actually been removed.

IX. References

- 
- [1] P. Maini, Z. Feng, A. Schwarzschild, Z. C. Lipton, and J. Z. Kolter, “TOFU: A Task of Fictitious Unlearning for LLMs,” in Conference on Language Modeling (COLM), 2024. arXiv:2401.06121.
 - [2] N. Li, A. Pan, A. Gopal, S. Yue, D. Berrios, et al., “The WMDP Benchmark: Measuring and Reducing Malicious Use with Unlearning,” in Proceedings of the 41st International Conference on Machine Learning (ICML), vol. 235, pp. 28525–28550, 2024.
 - [3] Z. Jin, P. Cao, C. Wang, Z. He, H. Yuan, J. Li, Y. Chen, K. Liu, and J. Zhao, “RWKU: Benchmarking Real-World Knowledge Unlearning for Large Language Models,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 37, 2024.
 - [4] Y. Zhang, F. Fan, Y. Zhang, Y. Yao, et al., “UnlearnCanvas: Stylized Image Dataset for Machine Unlearning Evaluation in Diffusion Models,” in NeurIPS 2024 Datasets and Benchmarks Track, 2024. arXiv:2402.11846.
 - [5] W. Shi, J. Lee, Y. Huang, S. Malladi, J. Zhao, A. Holtzman, D. Liu, L. Zettlemoyer, N. A. Smith, and C. Zhang, “MUSE: Machine Unlearning Six-Way Evaluation for Language Models,” in International Conference on Learning Representations (ICLR), 2025.
 - [6] V. Dorna, A. Reddy Mekala, W. Zhao, A. McCallum, Z. C. Lipton, J. Z. Kolter, and P. Maini, “OpenUnlearning: Accelerating LLM Unlearning via Unified Benchmarking of Methods and Metrics,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 38, 2025.
 - [7] S. Ramakrishna et al., “LUME: LLM Unlearning with Multitask Evaluations,” in SemEval-2025 Task 4, 2025. arXiv:2502.15097.
 - [8] K. Zhao, M. Kurmanji, G.-O. Barbulescu, E. Triantafillou, and P. Triantafillou, “What Makes Unlearning Hard and What to Do About It,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 37, pp. 12293–12333, 2024.
 - [9] A. Lynch, P. Guo, A. Ewart, S. Casper, and D. Hadfield-Menell, “Eight Methods to Evaluate Robust Unlearning in LLMs,” arXiv preprint arXiv:2402.16835, 2024.

- [10] J. Łucki, B. Wei, Y. Huang, P. Henderson, F. Tramèr, and J. Rando, “An Adversarial Perspective on Machine Unlearning for AI Safety,” arXiv preprint arXiv:2409.18025, 2024.
- [11] M. Doshi and B. Stickland, “Does Unlearning Truly Unlearn? A Black Box Evaluation of LLM Unlearning Methods,” arXiv preprint arXiv:2411.12103, 2024/2025.
- [12] S. Deeb and M. Roger, “Do Unlearning Methods Remove Information from Language Model Weights?” arXiv preprint arXiv:2410.08827, 2024.
- [13] L. Hayes, I. Shumailov, P. Triantafillou, A. Khalifa, and N. Papernot, “Inexact Unlearning Needs More Careful Evaluations to Avoid a False Sense of Privacy,” in IEEE SaTML, 2025. arXiv:2403.01218.
- [14] A. Thaker, S. Hu, A. Kale, S. Maurya, S. Wu, and N. Smith, “Position: LLM Unlearning Benchmarks are Weak Measures of Progress,” in IEEE SaTML, 2025. arXiv:2410.02879.
- [15] S. Jeon, J. Jeung, J. Kim, S. No, and H. Choi, “An Information Theoretic Evaluation Metric for Strong Unlearning,” in Proceedings of the AAAI Conference on Artificial Intelligence, 2026. arXiv:2405.17878.
- [16] J. Kim, J. Cha, and J. Kim, “Are We Truly Forgetting? A Critical Re-examination of Machine Unlearning Evaluation Protocols,” Neurocomputing, 2025/2026. arXiv:2503.06991.
- [17] Z. Feng, Y. E. Xu, A. Robey, R. Kirk, X. Davies, Y. Gal, A. Schwarzschild, and J. Z. Kolter, “Existing Large Language Model Unlearning Evaluations Are Inconclusive,” arXiv preprint arXiv:2506.00688, 2025.
- [18] S. Aerni, Y. Zhang, and F. Tramèr, “Evaluations of Machine Learning Privacy Defenses Are Misleading,” in Proceedings of the ACM Conference on Computer and Communications Security (CCS), 2024.
- [19] Y. Gao, A. Unal, A. Rangamani, and Z. Zhu, “An Illusion of Unlearning? Assessing Machine Unlearning Through Internal Representations,” in Proceedings of the 29th International Conference on Artificial Intelligence and Statistics (AISTATS), vol. 300, pp. 4114–4122, 2026.
- [20] A. A. Khan, H. Laga, and F. Sohel, “Metric Unreliability in Multimodal Machine Unlearning: A Systematic Analysis and Principled Unified Score,” arXiv preprint arXiv:2605.02206, 2026.
- [21] Y. Tu, P. Hu, and J. W. Ma, “A Reliable Cryptographic Framework for Empirical Machine Unlearning Evaluation,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 38, 2025.
- [22] Z. Wang, C. Zhang, Y. Chen, N. Baracaldo, S. R. Kadhe, and L. Yu, “Membership Inference Attacks as Privacy Tools: Reliability, Disparity and Ensemble,” in Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 1724–1738, 2025, doi: 10.1145/3719027.3744818.
- [23] Z. Di, S. Yu, Y. Vorobeychik, and Y. Liu, “Adversarial Machine Unlearning,” in International Conference on Learning Representations (ICLR), 2025.
- [24] C. Foster, J. Fogarty, D. Schoepf, M. Dugue, A. Öztireli, and A. Brintrup, “An Information Theoretic Approach to Machine Unlearning,” Transactions on Machine Learning Research (TMLR), 2025. arXiv:2402.01401.
- [25] L. Xue, S. Hu, W. Lu, Y. Shen, D. Li, P. Guo, Z. Zhou, M. Li, Y. Zhang, and L. Y. Zhang, “Towards Reliable Forgetting: A Survey on Machine Unlearning Verification,” ACM Computing Surveys, vol. 58, no. 12, Art. 314, 2026, doi: 10.1145/3807451.
- [26] T. Eisenhofer, D. Riepel, V. Chandrasekaran, E. Ghosh, O. Ohrimenko, and N. Papernot, “Verifiable and Provably Secure Machine Unlearning,” in IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), pp. 479–496, 2025, doi: 10.1109/SaTML64287.2025.00033.
- [27] J. Guo, Z. Zhao, H. Hou, W. Wang, and J. Jia, “Verifying in the Dark: Verifiable Machine Unlearning by Using Invisible Backdoor Triggers,” IEEE Transactions on Information Forensics and Security, vol. 19, pp. 708–721, 2024.

- [28] X. Gao, X. Ma, J. Wang, Y. Sun, B. Li, S. Ji, P. Cheng, and J. Chen, “VeriFi: Towards Verifiable Federated Unlearning,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 6, pp. 5720–5736, 2024, doi: 10.1109/TDSC.2024.3382321
- [29] C.-L. Wang, Q. Li, Z. Xiang, Y. Cao, and D. Wang, “Towards Lifecycle Unlearning Commitment Management: Measuring Sample-level Unlearning Completeness,” in *34th USENIX Security Symposium (USENIX Security 25)*, pp. 6481–6500, 2025.
- [30] J. Hu, W. Wang, Y. Dong, and L. Xue, “Learn What You Want to Unlearn: Unlearning Inversion Attacks Against Machine Unlearning,” in *IEEE Symposium on Security and Privacy (S&P)*, pp. 3257–3275, 2024.
- [31] E. Triantafillou, K. Kairouz, F. Pedregosa, L. Hayes, M. Kurmanji, K. Zhao, et al., “Are We Making Progress in Unlearning? Findings from the First NeurIPS Unlearning Competition,” *arXiv preprint arXiv:2406.09073*, 2024.
- [32] J. Geng, Q. Li, H. Woisetschlaeger, Z. Chen, F. Cai, Y. Wang, P. Nakov, H.-A. Jacobsen, and F. Karray, “A Comprehensive Survey of Machine Unlearning Techniques for Large Language Models,” *arXiv preprint arXiv:2503.01854*, 2025.
- [33] T. T. Nguyen, T. T. Huynh, Z. Ren, P. L. Nguyen, A. W.-C. Liew, H. Yin, and Q. V. H. Nguyen, “A Survey of Machine Unlearning,” *ACM Transactions on Intelligent Systems and Technology*, vol. 16, no. 5, Art. 108, 2025, doi: 10.1145/3749987.
- [34] U. N. Le-Khac and V. N. X. Truong, “A Survey on Large Language Models Unlearning: Taxonomy, Evaluations, and Future Directions,” *Artificial Intelligence Review*, vol. 58, Art. 399, 2025, doi: 10.1007/s10462-025-11376-7.
- [35] N. Li, C. Zhou, Y. Gao, H. Chen, Z. Zhang, B. Kuang, and A. Fu, “Machine Unlearning: Taxonomy, Metrics, Applications, Challenges, and Prospects,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 8, pp. 13709–13729, 2025, doi: 10.1109/TNNLS.2025.3530988.
- [36] S. Sehgal, A. Verma, and H. Bansal, “From Forgetting to Future: A Survey of Machine Unlearning Approaches,” *WIREs Data Mining and Knowledge Discovery*, vol. 16, no. 2, Art. e70082, 2026, doi: 10.1002/widm.70082.
- [37] W. Wang, Z. Tian, C. Zhang, and S. Yu, “Machine Unlearning: A Comprehensive Survey,” *arXiv preprint arXiv:2405.07406*, 2024.